Sentiment from Social Posts, Bar Plots, & Git

ISTA 131 Hw6

Context (read first)

In the late 2000s–2010s, Twitter crossed a historical threshold and became a primary venue for major political conversation—campaign messaging, live debate reactions, and rapid media narratives all played out in short posts. That shift makes social posts a useful (but imperfect) signal for quick, small-sample sentiment snapshots, provided we treat findings with healthy skepticism and focus on clean code, correct summaries, and faithful visuals rather than forecasting.

Introduction. This assignment gives you practice loading JSON data, performing text sentiment analysis with TextBlob, summarizing results in a pandas DataFrame, and visualizing them in a styled grouped bar chart. The data are small samples of scraped tweets mentioning two political figures scraped on two consecutive days in October 2023. Treat the results as a coding exercise, not a political forecast – lexicon-based sentiment on tiny samples is noisy by nature.

Instructions. Create a module named hw6.py. Below is the spec for four functions. Implement them and upload your module to the D2L Assignments folder.

Testing. Download hw6_test.py and put it in the same folder as your hw6.py module. Run it from the command line to see your current correctness score. You can examine the test module in a text editor to understand better what your code should do. The test module is part of the spec. The test file we will use to grade your program will be different and may uncover failings in your work not evident upon testing with the provided file. Add any necessary tests to make sure your code works in all cases.

Documentation. Your module must contain a header docstring containing your name, your section leader's name, the date, ISTA 131 Hw6, and a brief summary of the module. Each function must contain a docstring. Each function docstring should include a description of the function's purpose, the name, type, and purpose of each parameter, and the type and meaning of the function's return value.

Grading. Your module will be graded on correctness, documentation, and coding style. Code should be clear and concise. You will only lose style points if your code is a real mess. Include inline comments to explain tricky lines and summarize sections of code.

Collaboration. Collaboration is allowed. You are responsible for your learning. Depending too much on others will hurt you on the tests. "Helping" others too much harms them in reality. Cite any sources/collaborators in your header docstring. Leaving this out is dishonest.

Version Control. Hw6 is managed via GitHub Classroom. Click the Hw6 accept link posted on the D2L Assignment to create your private repository. Clone it to your machine – all data files and the test script are already included. Create hw6.py in the repo directory and work in it from there. Make at least 3 meaningful commits spaced over time, not one end-of-day dump. Suggested milestones:

1. initiating hw6.py
2. sentiment functions
3. plotting and main()

Use descriptive commit messages that say what changed (e.g., feat: implement get_sentiment with 0-0 filter). Avoid generic messages like update or final. There is no URL to submit – we access your repo directly and grade what is on GitHub, not what is on your machine.

Deductions: -10% for each missing meaningful commit (up to -30%); -5% for each non-descriptive commit message (up to -15%); -40% if the repo was not created through the accept link or no work is pushed.

Resources.

TextBlob documentation: https://textblob.readthedocs.io/en/dev/

pandas DataFrame visualization: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.html

pandas visualization user guide: https://pandas.pydata.org/pandas-docs/stable/user_guide/visualization.html

pandas DataFrame transpose: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.T.html

matplotlib named colors: https://matplotlib.org/examples/color/named_colors.html

matplotlib bar chart demo: https://matplotlib.org/examples/api/barchart_demo.html

If you don't have this document with you and you want to find all matplotlib pyplot functions, you can start the interpreter and type dir(plt) or pass the module to the built-in function dir. You will get a list of all the methods you might be interested in and then some. If you want more information on, say, errorbar, type help(plt.errorbar) into the interpreter. This functionality is available for any type (this is why we write docstrings).

Function specifications

In order to use TextBlob, you will need to install it from the command line:

```
Windows PowerShell
hw8> conda install -c conda-forge textblob
Collecting package metadata (repodata.json): done
Solving environment: done
```

If possible, use conda to install it, not pip.

get_sentiment: This function takes a filename as its sole argument. The file contains a JSON string representing a list of the text of a number of tweets. Open the file in read mode and use the json module to load its contents into a Python list. Use TextBlob to perform sentiment analysis on each of the tweets, rejecting any that have both polarity and subjectivity equal to 0.0. Return a two-element list containing the mean of the remaining polarities and the sample standard deviation of those polarities.

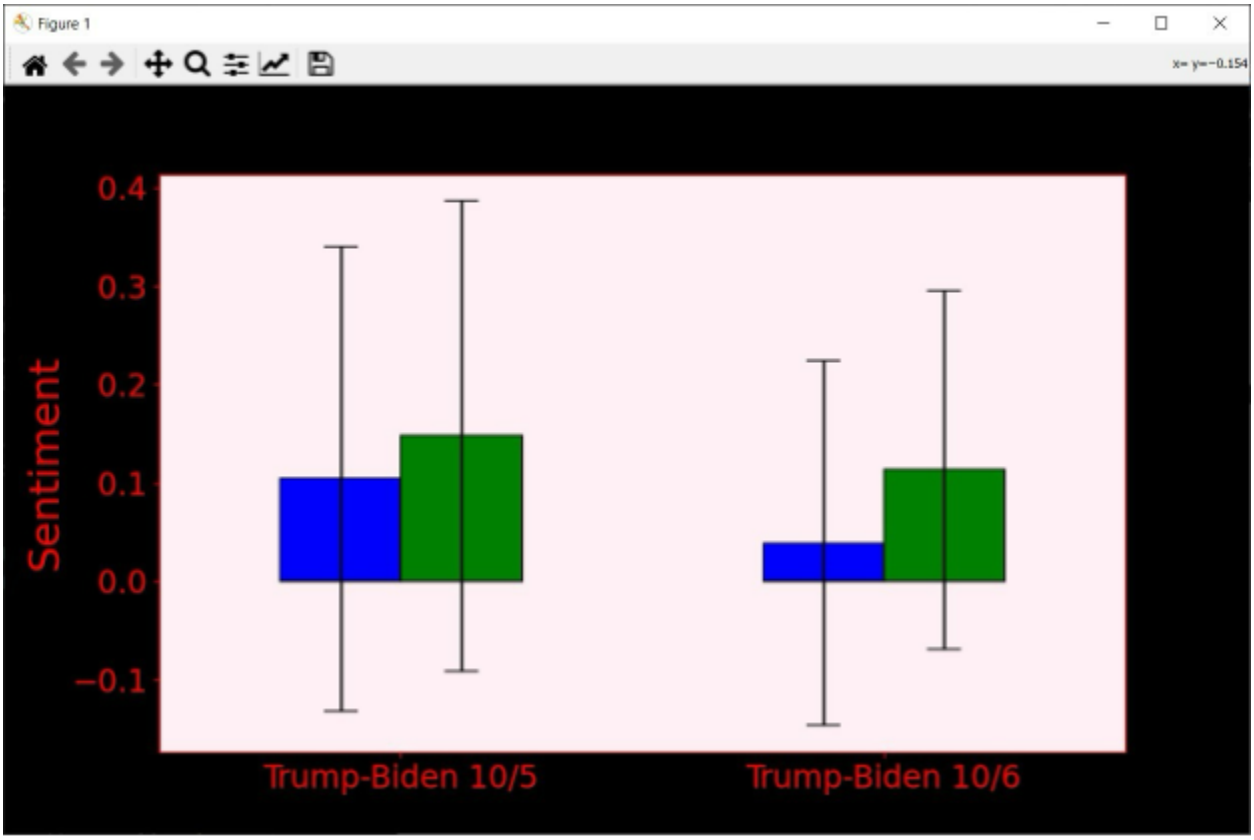
get_ct_sentiment_frame: This function takes no arguments and returns a DataFrame that looks like the image below. The columns contain the mean and standard deviation of the polarities of the tweet sample for each candidate scraped on the two days as determined by get_sentiment. Do not hard-code values – your function must call get_sentiment on each of the four JSON files in the data directory and assemble the results.

	10/5 mean	10/5 std	10/6 mean	10/6 std
Trump	0.104820	0.236196	0.039278	0.185205
Biden	0.148314	0.238706	0.113722	0.182319

make_fig: This function takes a sentiment frame (the return value from get_ct_sentiment_frame) and constructs and returns a grouped bar chart with two bars per candidate (one for each day's mean polarity) and vertical error bars using the standard deviations. The function must create the full figure object and return it without displaying or saving it. The rubric for this figure:

- no points unless the sentiment frame passes its test

- +3 each bar (+12 total): correct size (+1) and color (+1).
- +6 error bars: all have correct vertical size (+3) and cap size (+3).
- +1 each (+3 total): x and y tick labels, y label are correct values.
- +1 each (+3 total): x and y tick labels, y label are correct sizes (tick labels 18, y label 24).
- +1 each (+3 total): x and y tick labels, y label are correct color.
- +4: spine colors are correct.
- +4: figure face color is correct.
- +3: plot background color is almost correct.
- +5: plot background color is exactly correct.
- -3: plot needs resizing to grade.



main: This function takes no arguments. Get the sentiment frame, print a blank line, print the frame, make the figure via make_fig, and display it with plt.show(). While main is not directly scored, errors here can prevent the figure from being graded.